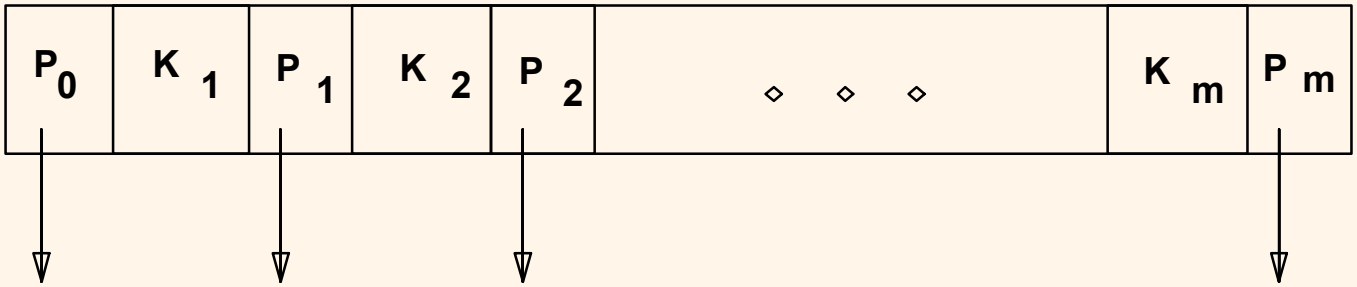
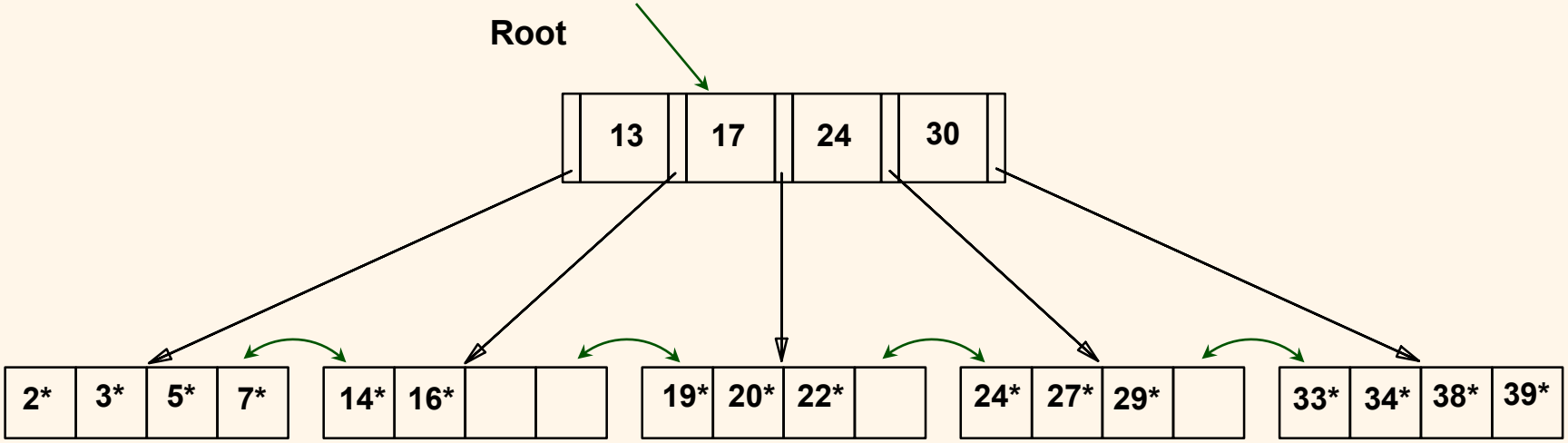
B+ Tree

- Minimum 50% occupancy for each node except the root
- Each node contains ***m*** entries where
  - **d** <= *m* <= 2**d**, **d** = *order* of the tree
  - m+1 pointers to children
    - Pointer P_i points to subtree where key values $K_i <= K < K_{i+1}$
    - P_0 points to tree where $K < K_1$
    - P_m points to tree where $K >= K_m$



Example B+ Tree

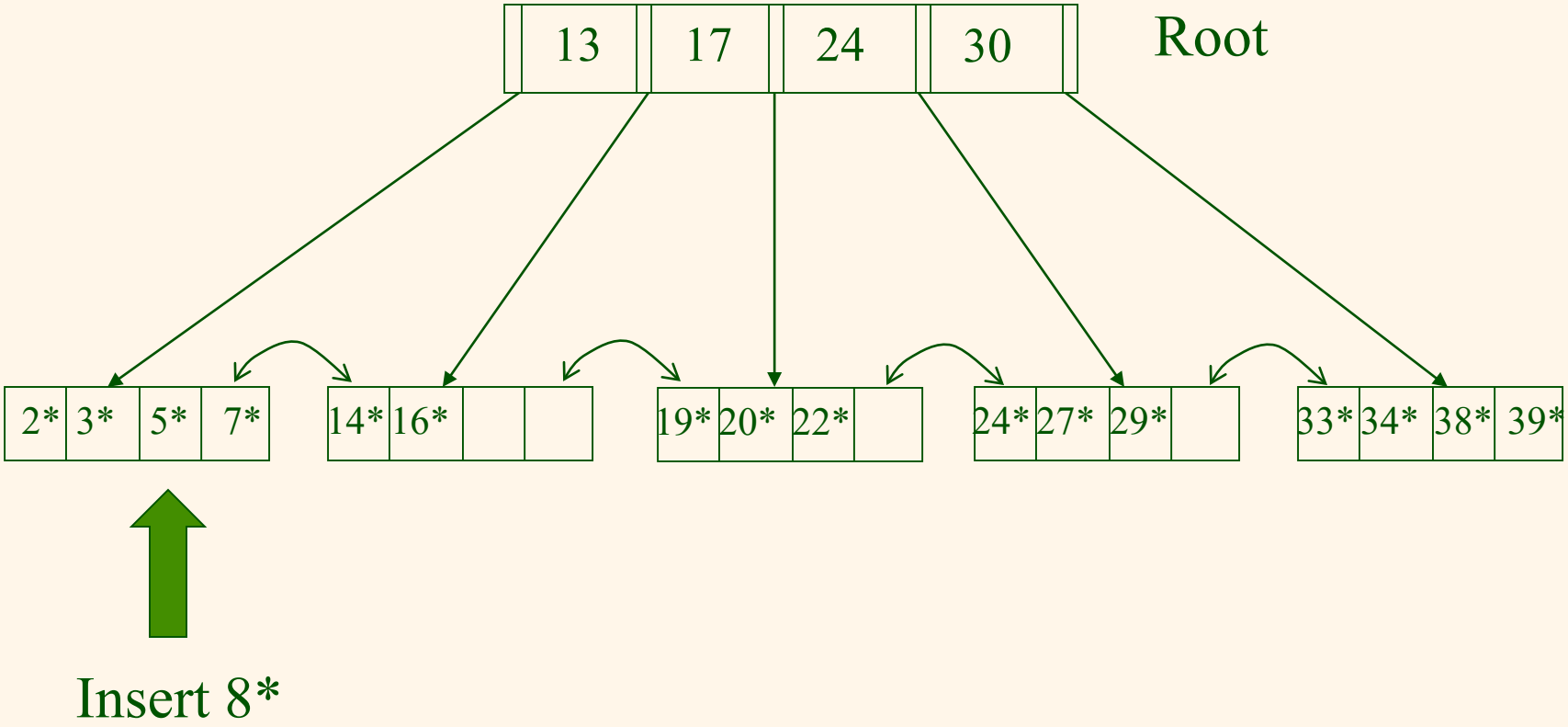
- Search begins at root, and key comparisons direct it to a leaf (as in ISAM).
- E.g., find 5*, 15*, all data entries 20<=k<27 ...



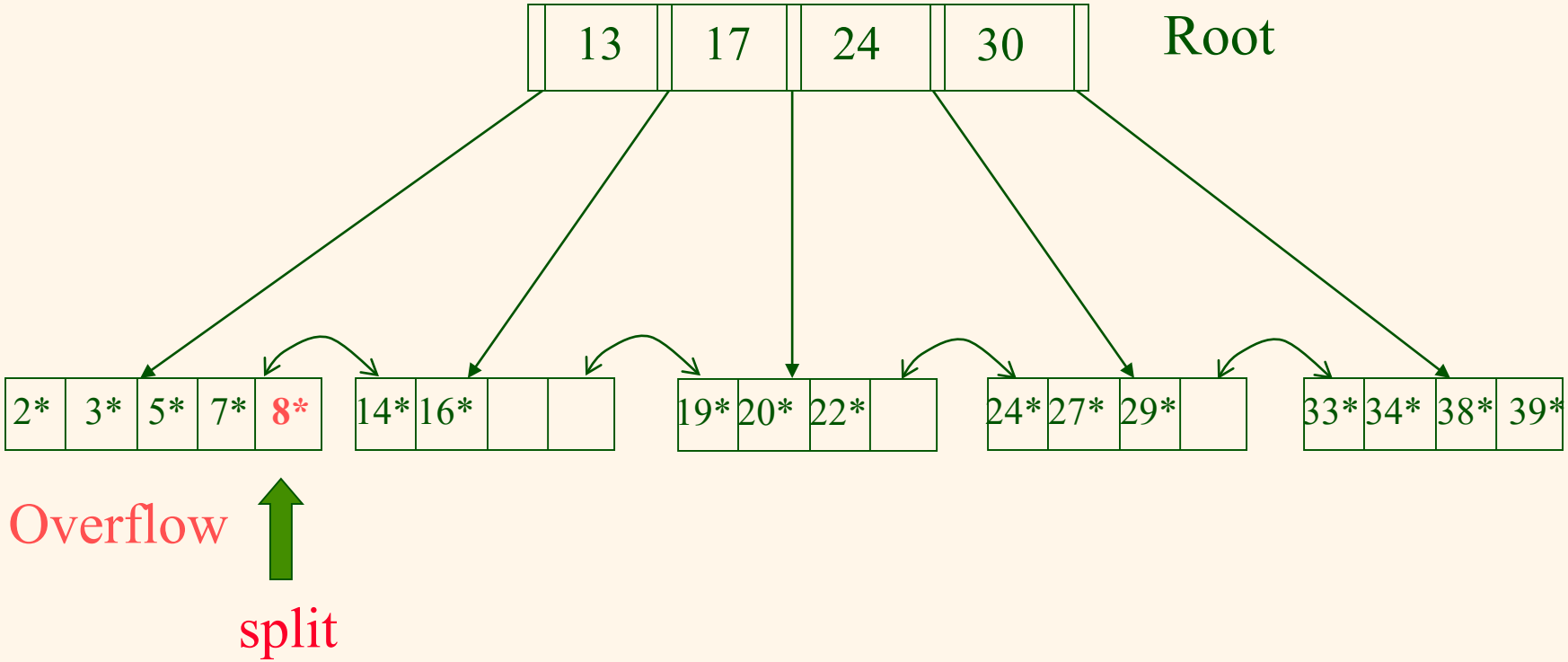
Inserting a Data Entry into a B+ Tree

- Find correct leaf *L*
- Put data entry onto *L*
  - If *L* has enough space, *done*!
  - Else, *split* *L into L and a new node L2*
    - Redistribute entries evenly, **copy up** middle key
    - Insert index entry pointing to *L2* into parent of *L*
- This can happen recursively
  - To split index node, redistribute entries evenly, but **push up** middle key (Contrast with leaf splits)
- Splits "grow" tree - root split increases height

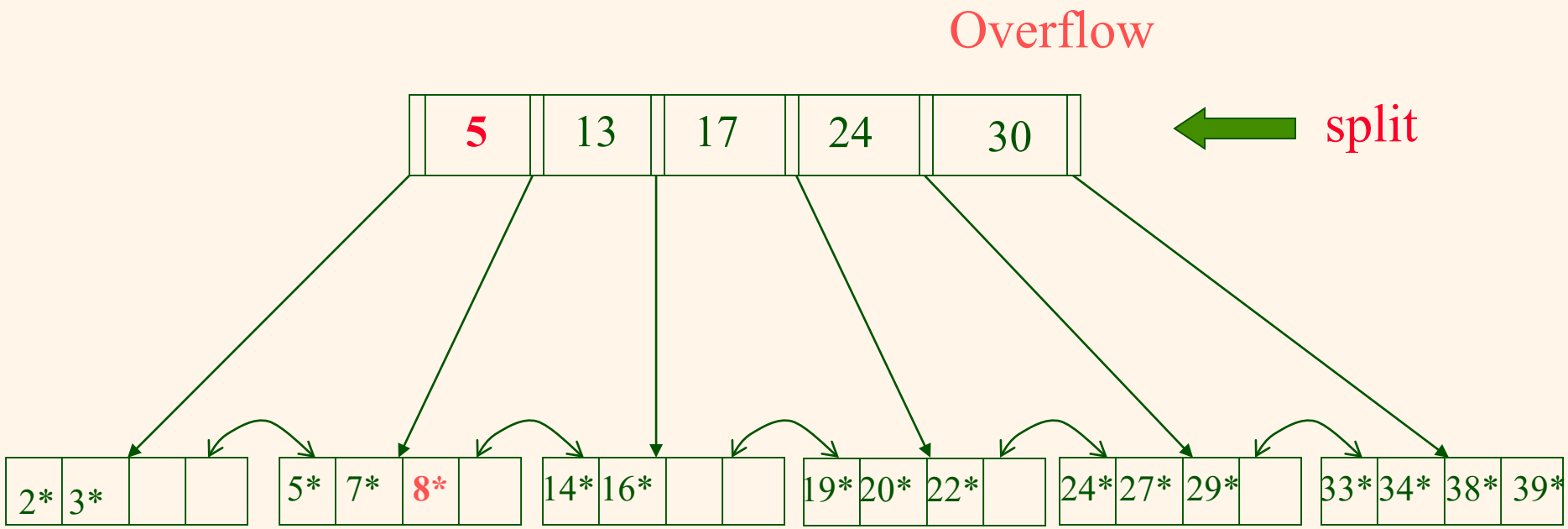
Inserting 8 (1)*



Inserting 8 (2)*

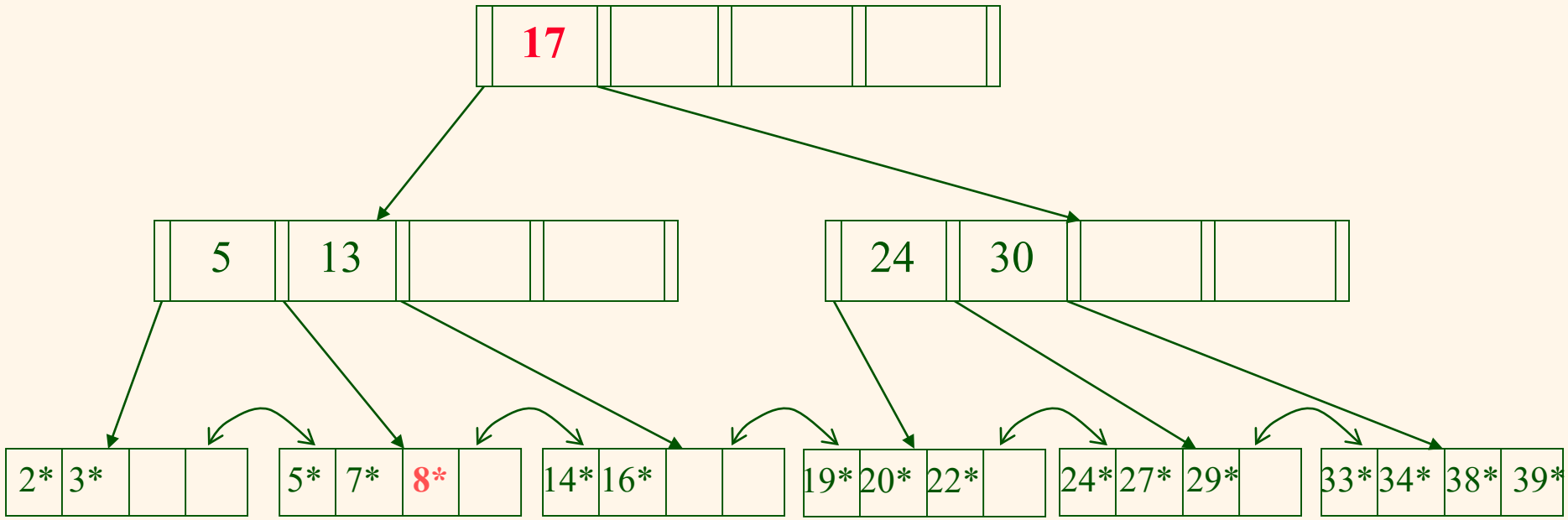


Inserting 8 (3)*



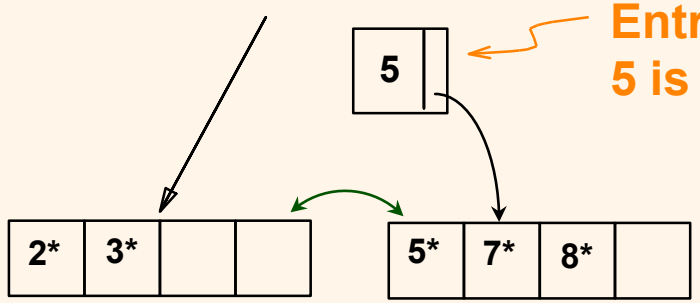
Key 5 (in leaf node) is "copied up" into parent node.

Inserting 8 (4)*



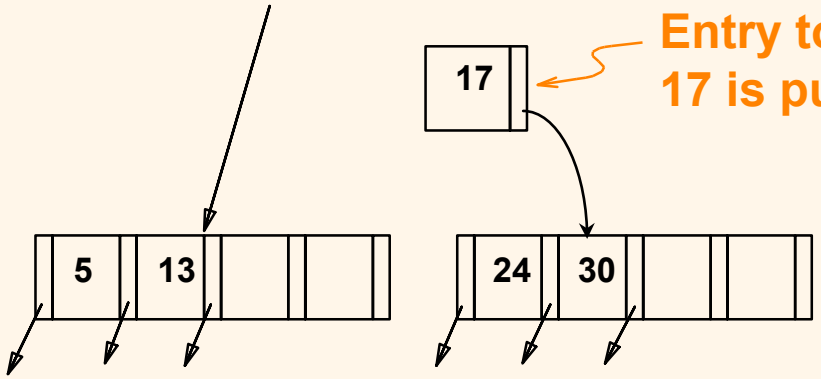
Key 17 (in index node) is "pushed up" into parent node.

"Copy Up" vs. "Push Up"



Entry to be inserted in parent node:
5 is copied up and still appear in leaf

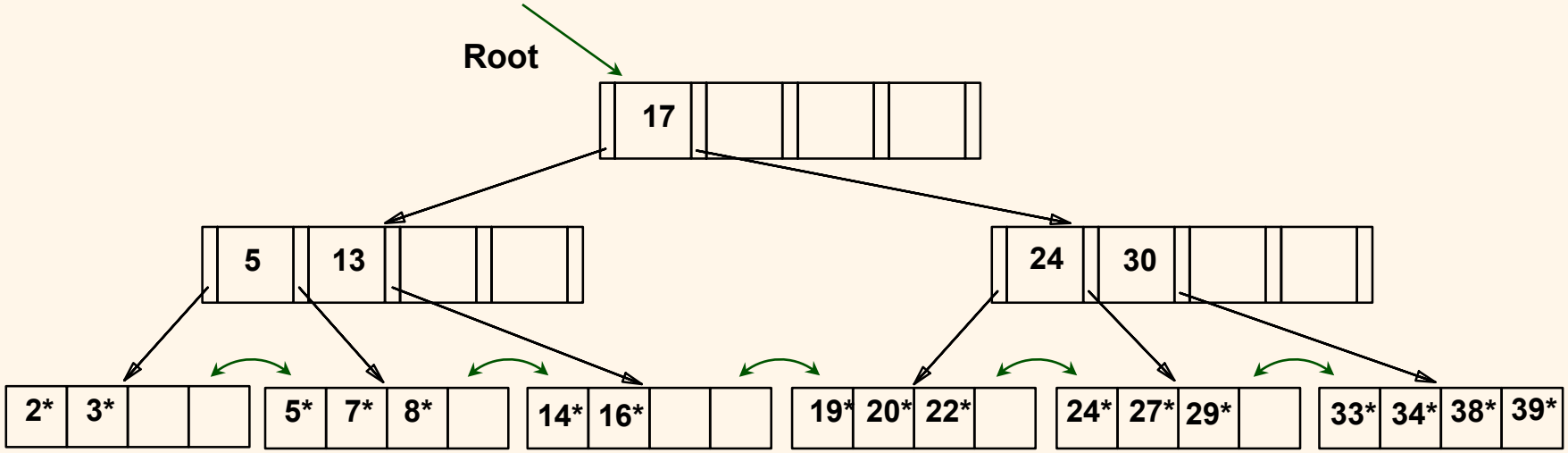
Leaf node split



Entry to be inserted in parent node:
17 is pushed up and appears once in index

Index node split

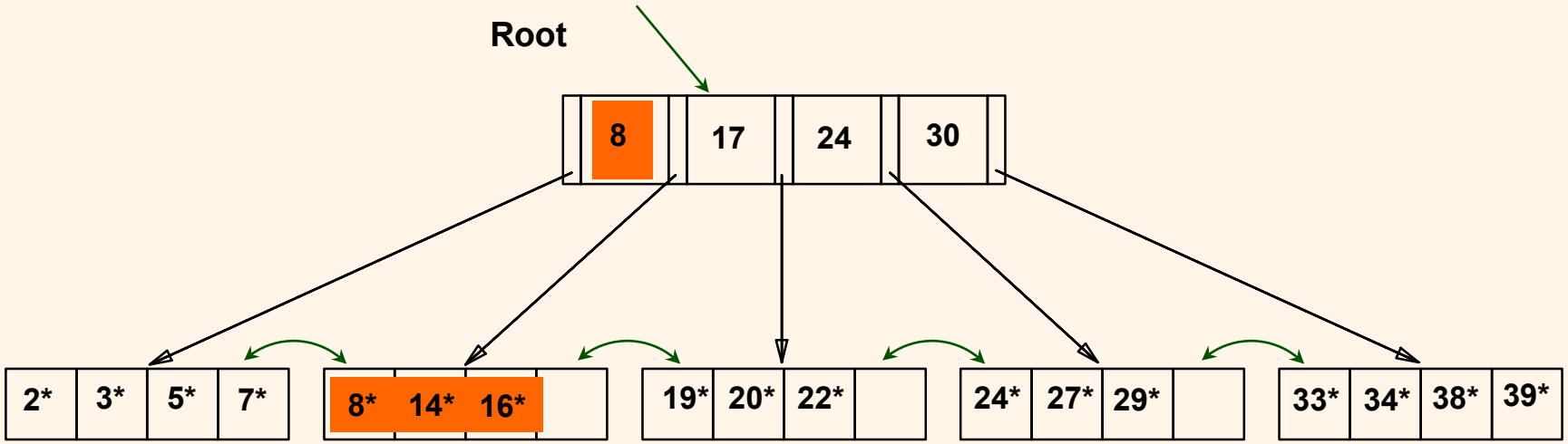
After Inserting 8 into B+ Tree*



- Root has been split, leading to height increase
- Can avoid split by re-distributing entries

Inserting 8 Using Re-distribution*

- Check neighbor(s) to see if it's feasible to re-distribute

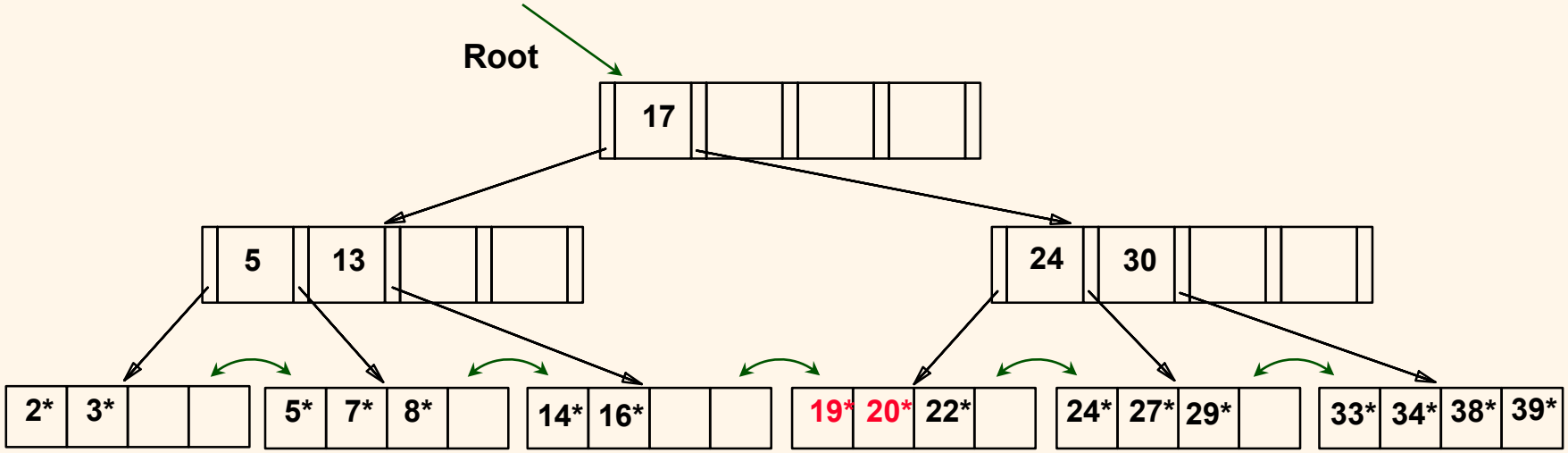


- In most versions, the re-distribution technique is not applied during insertions.

Deleting a Data Entry from a B+ Tree

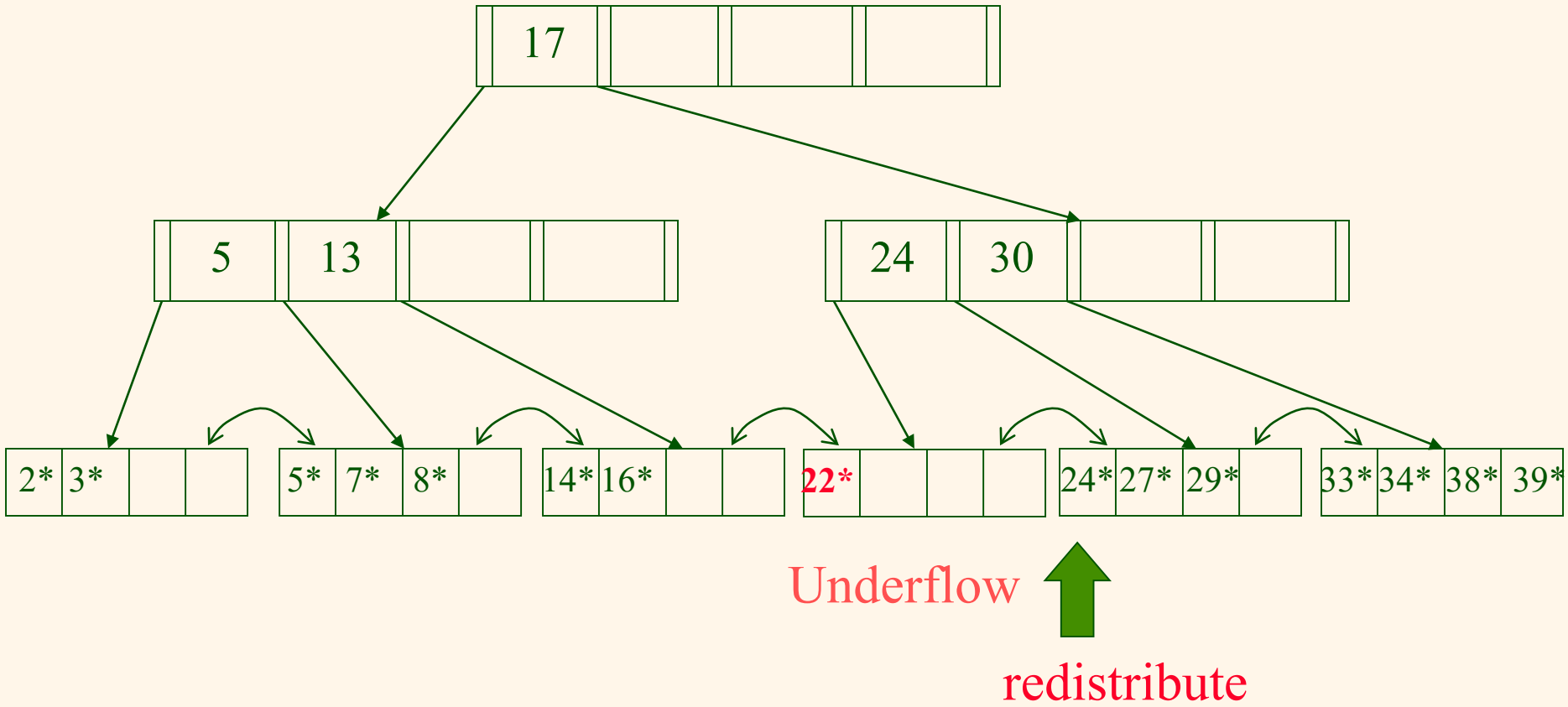
- Start at root, find leaf *L* where entry exist
- Remove the entry
  - If L is at least half-full, *done!*
  - If L has only **d-1** entries,
    - Try to re-distribute by borrowing from *sibling* *(adjacent node with same parent as L)*
    - If re-distribution fails, *merge* *L* and sibling
    - If merge occurred, must delete entry (pointing to *L* or sibling) from parent of *L*
- Merge could propagate to root, decreasing height of tree

Want to Delete 19 and 20* ...*

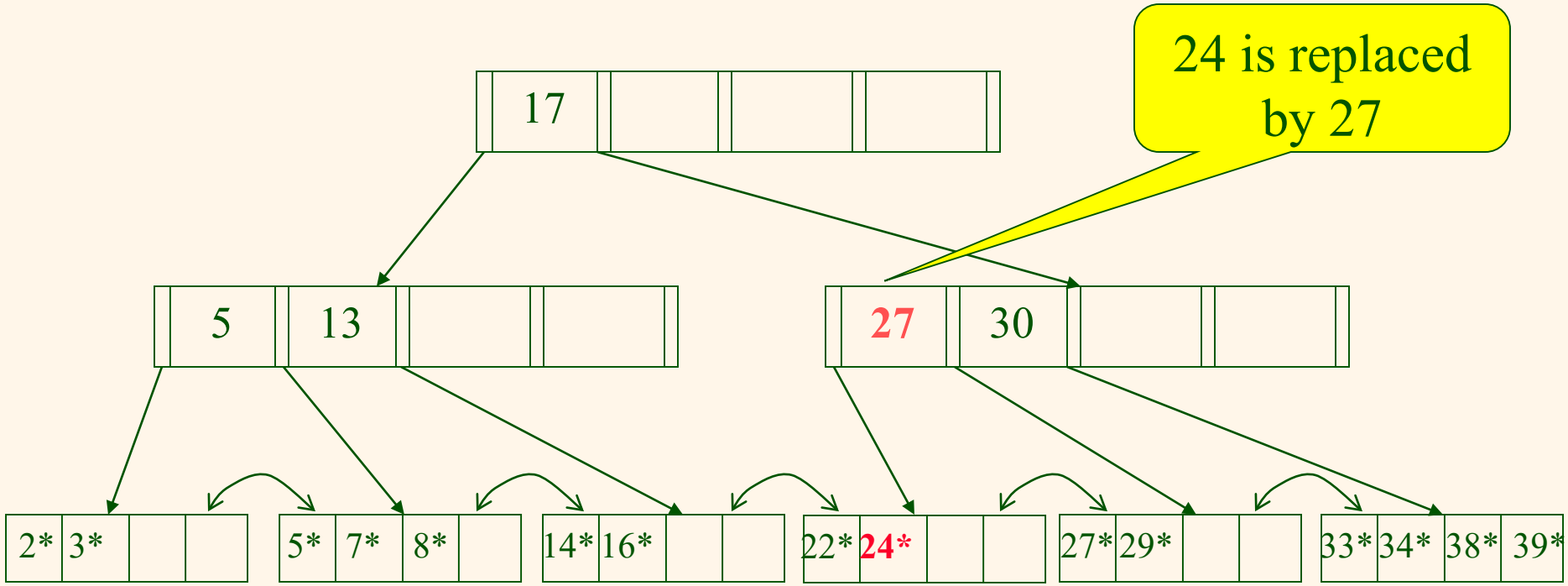


- Deleting 19* is easy
- Deleting 20* is done with re-distribution
  - middle key is *copied up*

Deleting 19, 20**

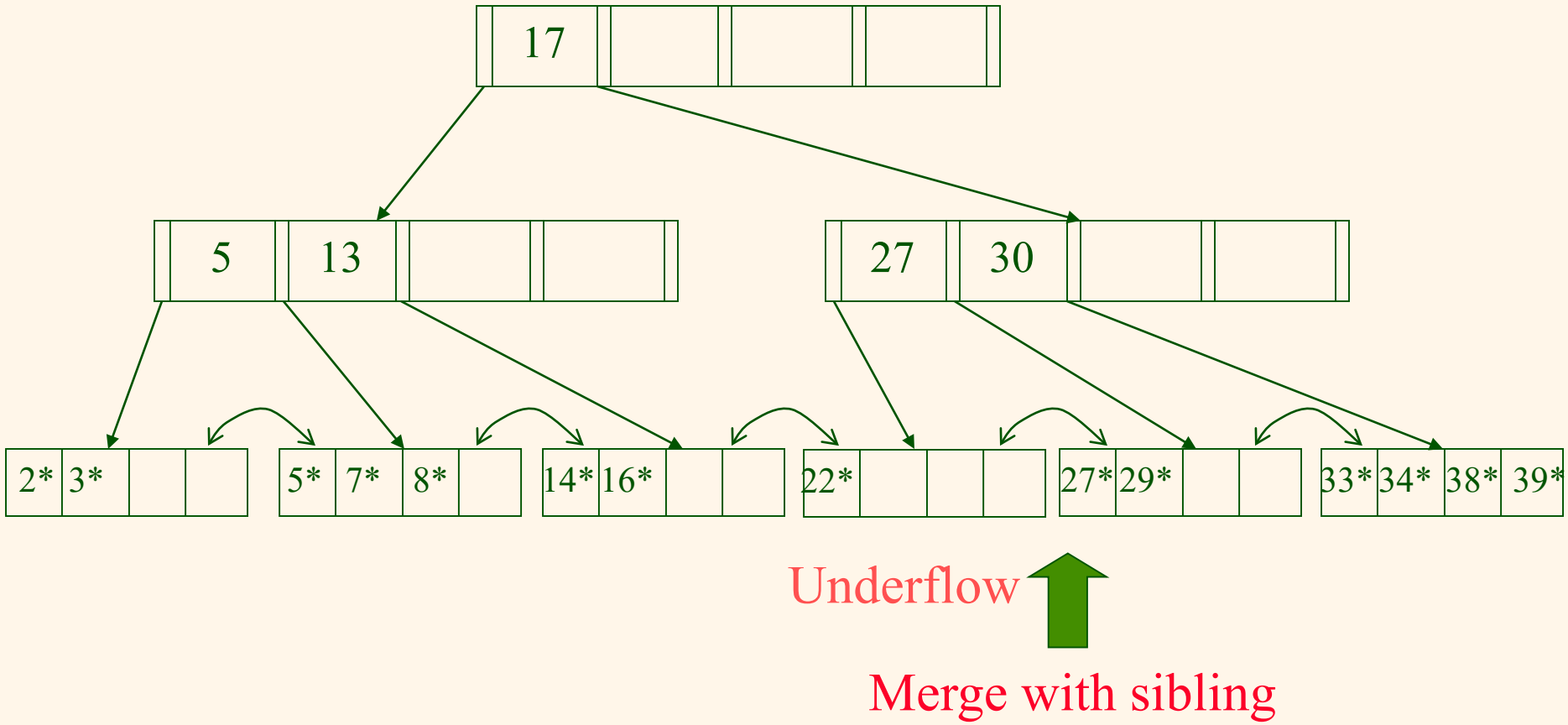


After Deleting 19, 20**

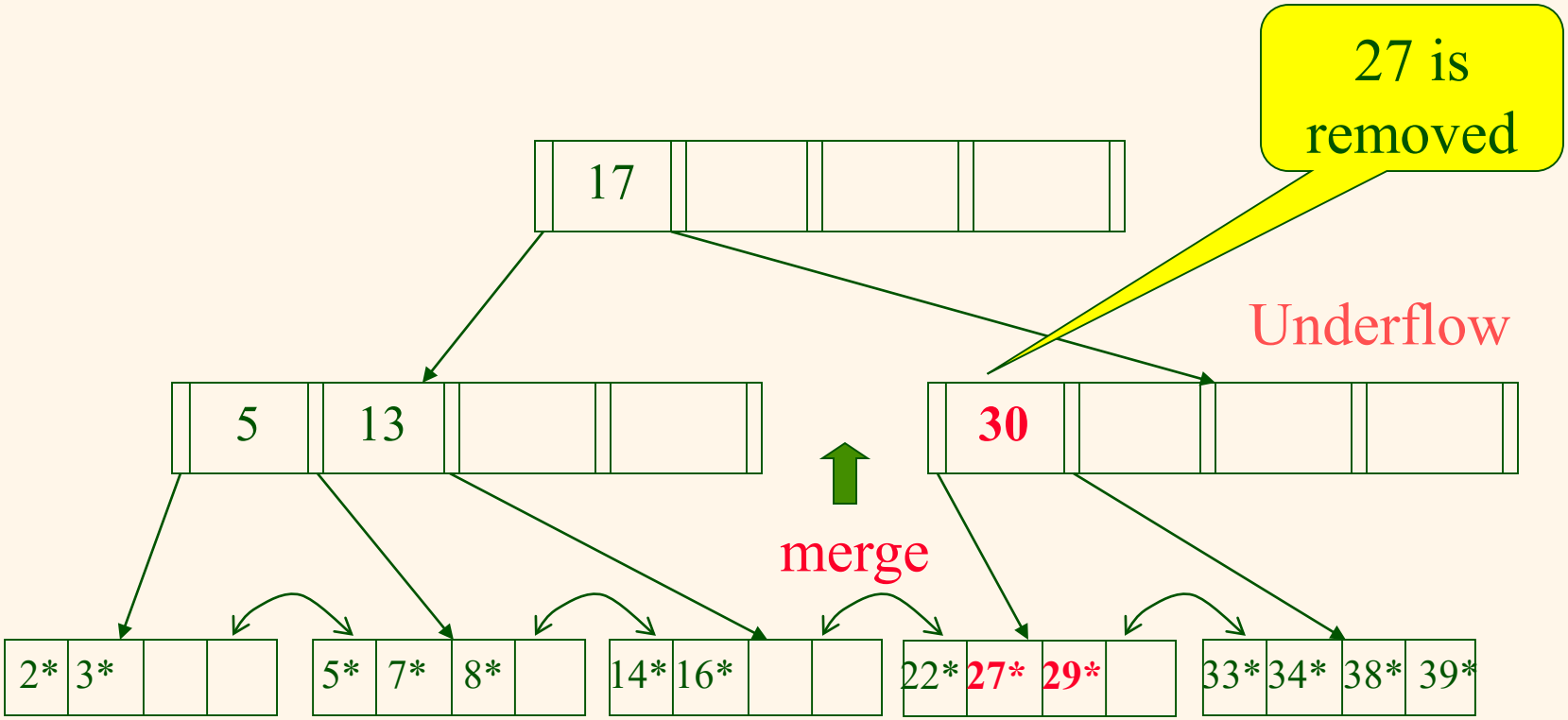


Now want to delete 24*

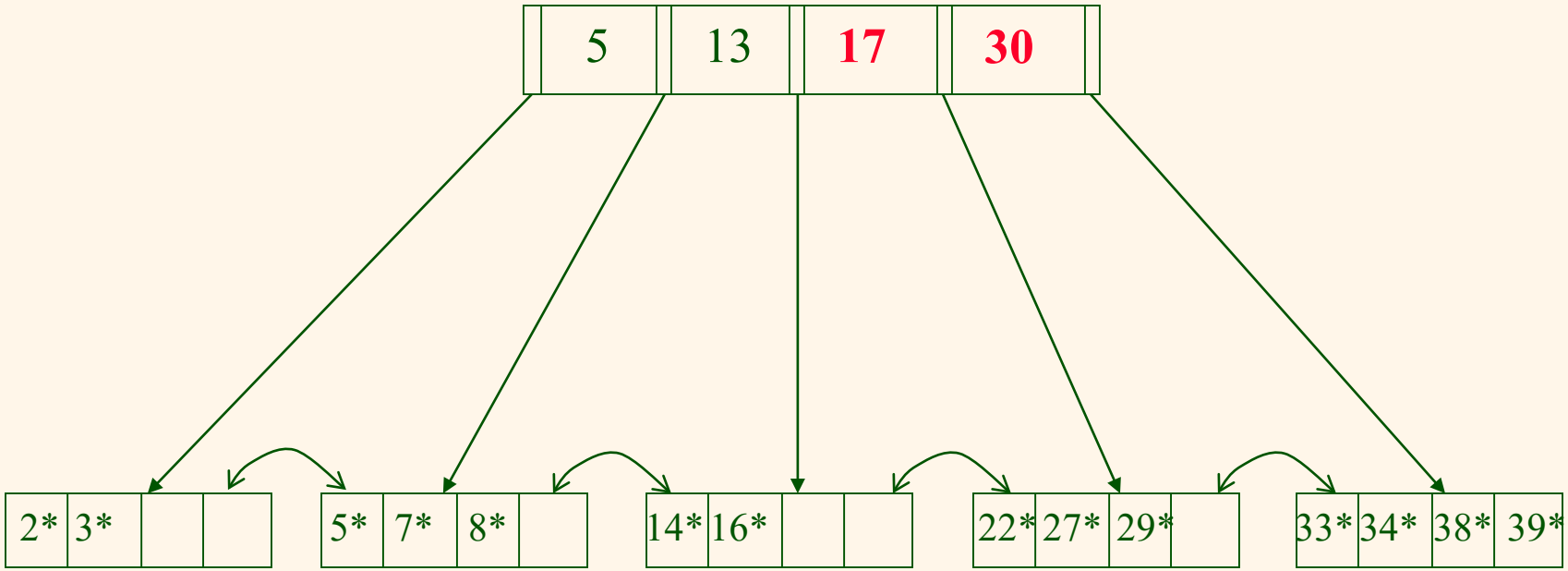
Deleting 24 (1)*



Deleting 24 (2)*



Deleting 24 (3)*

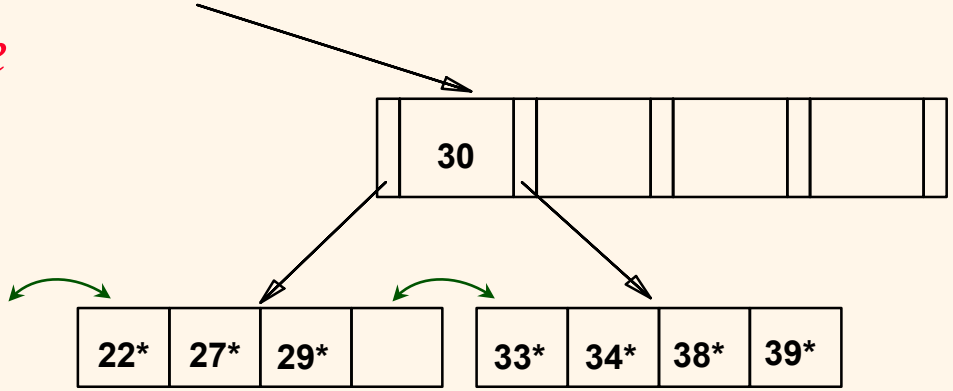


- After deleting 24*, the tree shrinks.

"Delete" vs. "Pull Down"

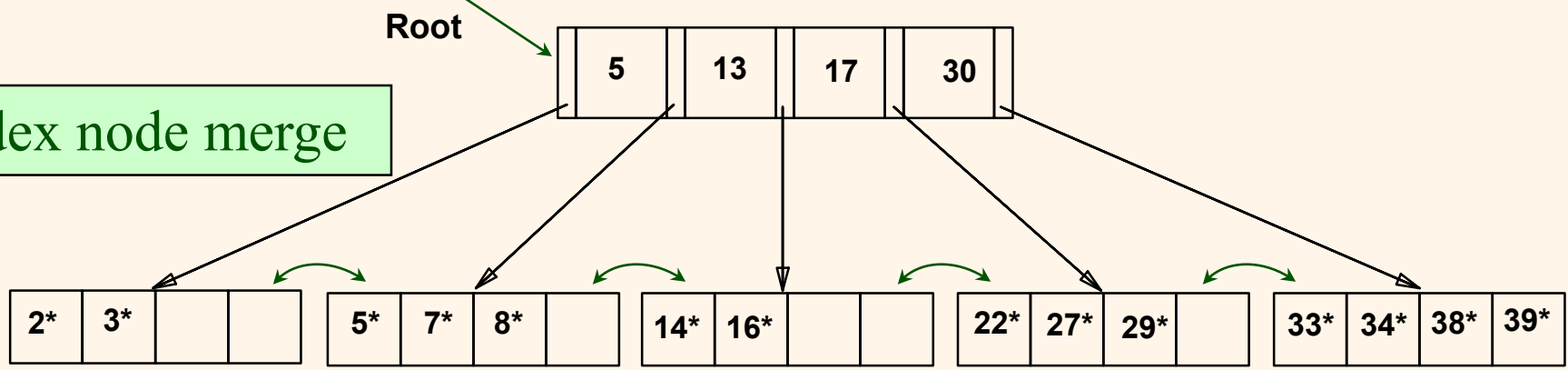
- Merge leaf nodes, *delete* index entry in parent

Leaf node merge



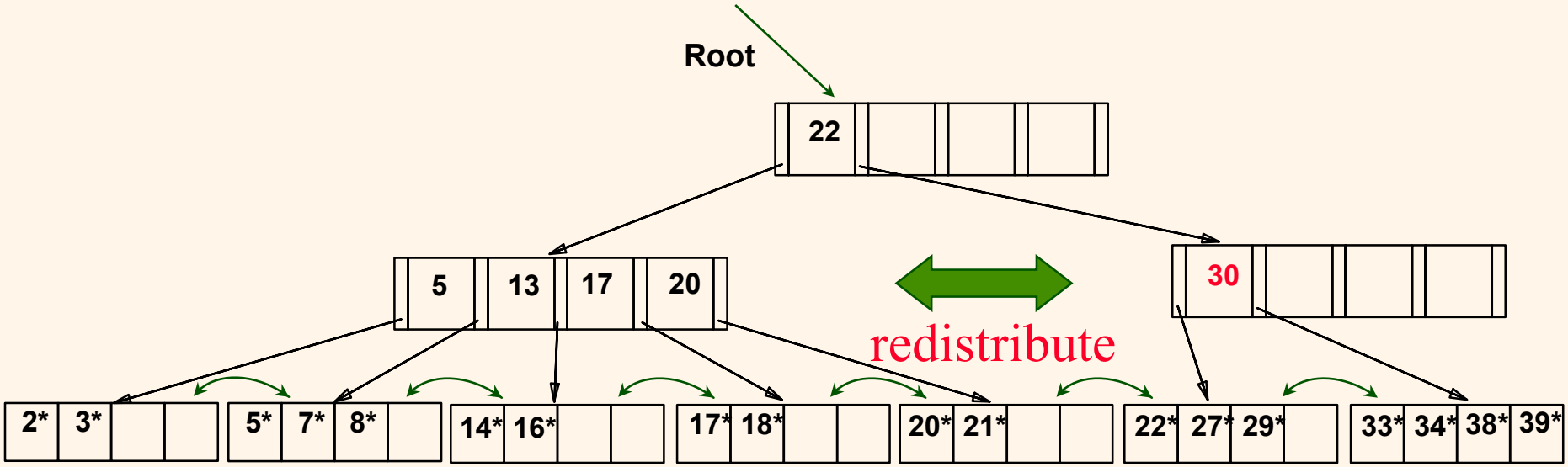
- Merge index nodes, *pull down* index entry in parent

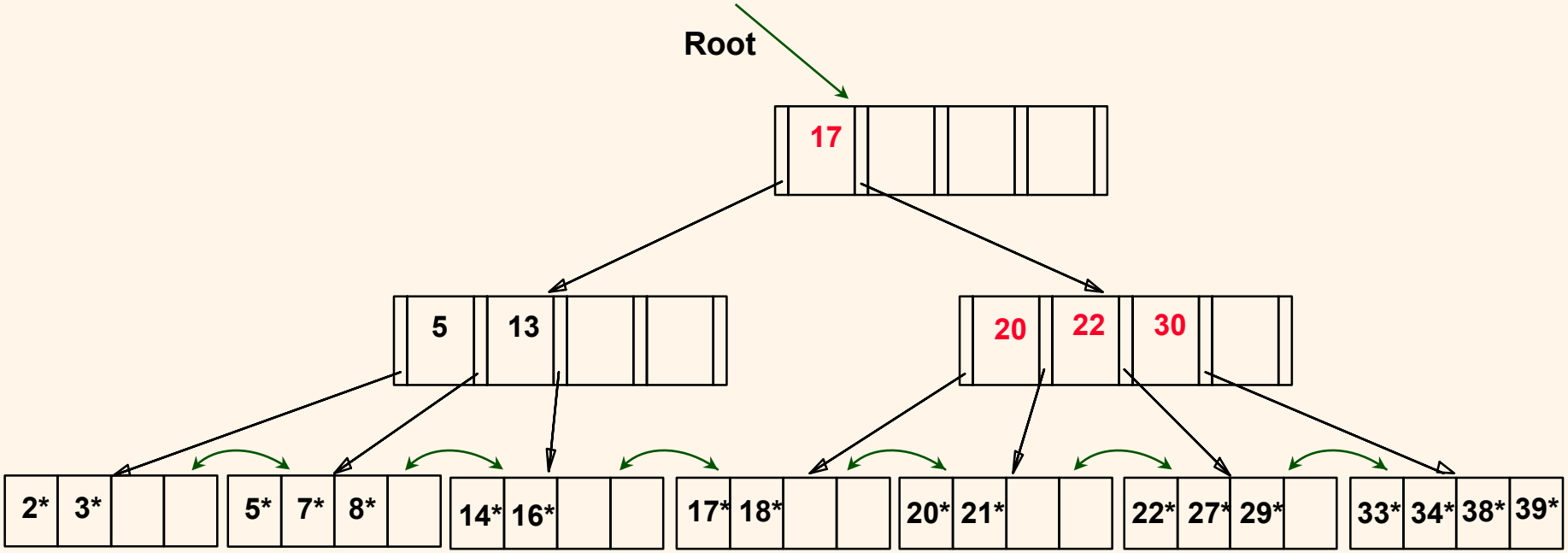
Index node merge



Example of Non-leaf Re-distribution

- Tree is shown below *during deletion* of 24*. (What could be a possible initial tree?)
- In contrast to previous example, can re-distribute entry from left child of root to right child.



After Re-distribution

- Intuitively, entries are re-distributed by *`pushing through'* the splitting entry in the parent node.
- It suffices to re-distribute index entry with key 20; we've re-distributed 17 as well for illustration.